

CÁC METRICS ĐÁNH GIÁ MÔ HÌNH VÀ LUỒNG HOẠT ĐỘNG

1. Các bộ dữ liệu (Datasets) tham khảo

1.1. Tập dữ liệu Spider

- **Bài báo nền tảng:** *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task* (Yu et al., EMNLP 2018).
- **Link:** [arXiv:1809.08887](https://arxiv.org/abs/1809.08887)
- **Nội dung:**
 - **Database thực tế (.sqlite):** Chứa 200 bộ cơ sở dữ liệu từ nhiều domain khác nhau, bao gồm cả dữ liệu mẫu tương ứng. Kích thước database khá nhỏ, chủ yếu để test sự am hiểu cấu trúc (Schema).
 - **Bộ Question-Query:** Hơn 10.000 cặp câu hỏi tự nhiên và câu lệnh SQL tương ứng, được đánh nhãn thủ công với các mức độ khó từ Dễ (Easy) đến Cực khó (Extra Hard).

1.2. Tập dữ liệu BIRD

- **Bài báo nền tảng:** *Can LLM Already Serve as A Database Interface? A Big Benchmark for Large-Scale Database Grounded Text-to-SQLs* (Li et al., NeurIPS 2023).
- **Link:** [arXiv:2305.03111](https://arxiv.org/abs/2305.03111)
- **Nội dung:**
 - **Database thực tế (.sqlite):** Chứa 95 bộ cơ sở dữ liệu với tổng kích thước lên tới hàng chục GB từ 37 domain khác nhau (tài chính, y tế, giáo dục...). Dữ liệu mang tính thực tế cao, chứa các giá trị "bẩn" (dirty data), nhiễu và đòi hỏi phải hiểu sâu về giá trị dữ liệu bên trong (Database Values) chứ không chỉ dừng ở cấu trúc (Schema).
 - **Bộ Question-Query:** 12.751 cặp câu hỏi tự nhiên và câu SQL, yêu cầu phân tích suy luận phức tạp và tối ưu hóa thời gian chạy. Điểm đánh giá bao gồm cả độ chính xác thực thi (EX) và hiệu suất thực thi (VES).
 - **Bộ Kiến thức bổ sung (External Knowledge/Evidence):** Mỗi bộ CSDL trong BIRD đi kèm với các file tài liệu mô tả (giải thích từ viết tắt, định nghĩa các cột, các công thức toán học nghiệp vụ riêng của domain đó) nhằm cung cấp ngữ cảnh giống hệt như một Data Analyst thực tế nhận được khi làm việc với CSDL của doanh nghiệp.

2. Các Metrics đánh giá (Evaluation Metrics)

2.1. Exact Match (EM) & Execution Accuracy (EX)

Hai chỉ số này là tiêu chuẩn vàng để đo lường độ chính xác của các mô hình sinh mã SQL.

- **Bài báo nền tảng:** *Spider* (Yu et al., EMNLP 2018).
- **Link:** [arXiv:1809.08887](https://arxiv.org/abs/1809.08887)

a) Exact Match (EM)

- **Công thức:**

$$EM = \prod_{c \in C} \mathbb{I}(\text{Set}(P_c) == \text{Set}(G_c))$$

C là tập các mệnh đề SQL như *SELECT*, *WHERE*, *GROUP BY*;

P_c là tập hợp thành phần dự đoán;

G_c là tập hợp gốc;

hàm chỉ thị $\mathbb{I}$ trả về 1 nếu trùng khớp, ngược lại là 0.

- **Tác dụng:** Đo lường xem câu SQL do AI sinh ra có chứa cấu trúc và các thành phần y hệt câu SQL mẫu (đáp án) hay không. Metric này so sánh bằng "tập hợp" (Set) nên không bị bắt bẻ về mặt thứ tự chữ.
- **Ví dụ:**
 - Câu mẫu: **SELECT name FROM users WHERE age > 18 AND city = 'Hanoi'**
 - AI sinh ra: **SELECT name FROM users WHERE city = 'Hanoi' AND age > 18**
 - => *Kết quả:* Dù thứ tự điều kiện WHERE bị đảo ngược, thuật toán chia tập hợp vẫn nhận ra 2 điều kiện này giống hệt nhau. Điểm EM = 1 (Chính xác).

b) Execution Accuracy (EX)

- **Công thức:**

$$EX = \begin{cases} 1, & \text{nếu } \text{Result}(S_{pred}, D) = \text{Result}(S_{gold}, D) \\ 0, & \text{nếu khác} \end{cases}$$

- **Tác dụng:** Không quan tâm AI viết code dài ngắn hay cấu trúc lạ thế nào, chỉ cần mang câu lệnh đó chạy trên database thật (D), nếu bảng kết quả trả về giống hệt bảng kết quả của câu mẫu thì được tính là đúng.
- **Ví dụ:** AI dùng lệnh JOIN có cú pháp khác hoàn toàn với đáp án, nhưng khi bấm nút chạy, nó vẫn xuất ra đúng danh sách "5 khách hàng mua nhiều nhất". => Điểm EX = 1 (Chạy đúng).

```
SELECT name FROM employees WHERE salary = (SELECT MAX(salary) FROM employees)
```

```
SELECT name FROM employees ORDER BY salary DESC LIMIT 1
```

```
SELECT users.name FROM users JOIN admins ON users.id = admins.user_id
```

```
SELECT name FROM users JOIN admins ON users.id = admins.user_id
```

2.2. Valid Efficiency Score (VES)

- **Bài báo nền tảng:** *BIRD* (Li et al., NeurIPS 2023).
- **Link:** [arXiv:2305.03111](https://arxiv.org/abs/2305.03111)
- **Công thức:**

$$VES = \frac{1}{N} \sum_{i=1}^N V_i \times \sqrt{\frac{R_{gold,i}}{R_{pred,i}}}$$

$V_i \in \{0, 1\}$ đại diện cho câu có chạy ra đúng kết quả hay không;

R đại diện cho thời gian chạy.

Cần bậc hai giúp làm mượt điểm số cho các truy vấn chạy quá nhanh.

- **Tác dụng:** Kết hợp đánh giá cả "Độ chính xác" (Valid) và "Tốc độ" (Efficiency). Metric này phạt các câu lệnh SQL viết đúng kết quả nhưng lại quét toàn bộ bảng một cách lãng phí, gây chậm hệ thống.
- **Ví dụ:**
 - Câu SQL mẫu (đáp án) chạy mất **1 giây**.
 - AI viết một câu lờn nhau quá rườm rà, chạy ra đúng kết quả nhưng mất tới **4 giây**.
 - => *Kết quả:* Điểm VES của câu này bị giảm xuống chỉ còn một nửa so với điểm tối đa (vì bị chia cho $\sqrt{4/1} = 2$).

2.3. Schema Linkage Accuracy

- **Bài báo nền tảng:** *RAT-SQL* (Wang et al., ACL 2020).
- **Link:** [arXiv:1911.04942](https://arxiv.org/abs/1911.04942)
 - **Công thức:**

$$Accuracy_{SL} = \frac{|E_{pred} \cap E_{gold}|}{|E_{gold}|}$$

E là tập hợp các Entity bao gồm tên Table và tên Column.

- **Tác dụng:** Đo lường khả năng AI tìm và nối đúng từ vựng trong câu hỏi (ngôn ngữ tự nhiên) với tên cột/bảng (thực thể vật lý) trong Database.
- **Ví dụ:**
 - Người dùng hỏi: "Tìm số điện thoại của sinh viên".
 - Nếu AI nhận diện đúng từ "số điện thoại" trở vào cột contact_number và "sinh viên" trở vào bảng Students, AI đạt độ chính xác 100% ở khâu này. Đây là bước tiền đề quan trọng trước khi sinh ra câu lệnh SQL.

2.4. Query Latency & Cost Reduction Ratio (CRR)

- **Bài báo nền tảng:** Bao: *Making Learned Query Optimization Practical* (Marcus et al., SIGMOD 2021).
- **Link:** [arXiv:2004.03811](https://arxiv.org/abs/2004.03811)
- **Công thức Latency (Thời gian thực thi trung bình để loại nhiễu):**

$$Latency = \frac{1}{k} \sum_{j=1}^k \Delta t_j$$

- **Công thức Cost Reduction Ratio (Tỷ lệ giảm chi phí):**

$$CRR = \frac{Cost_{pg} - Cost_{model}}{Cost_{pg}}$$

- **Tác dụng:** CRR đo lường % chi phí tài nguyên phần cứng (CPU, I/O) hoặc % thời gian chạy hệ thống tiết kiệm được sau khi AI tối ưu câu SQL.
- **Ví dụ:** Câu truy vấn gốc (do người dùng viết) tốn **100 đơn vị** chi phí để chạy. Qua bộ tối ưu của AI viết lại, câu lệnh chỉ còn tốn **30 đơn vị**. Tỷ lệ tiết kiệm CRR lúc này là $(100 - 30)/100 = 70\%$.

2.5. SQL Complexity Score / SQL Hardness Criteria

- **Nguồn gốc:** Phụ lục bài báo *Spider (2018)*.
- **Công thức tính điểm (Ví dụ):**

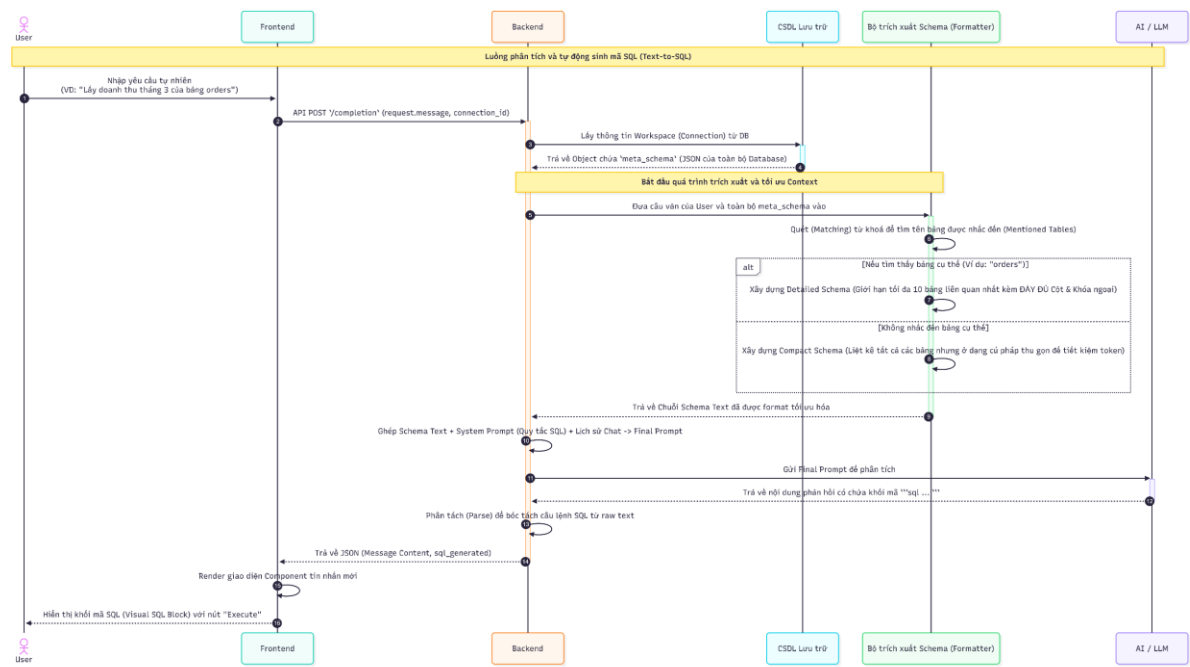
$$Score = w_1 N_{SELECT} + w_2 N_{WHERE} + w_3 N_{GROUPBY} + w_4 N_{JOIN} + \dots$$

(Ví dụ trọng số quy ước: $JOIN = 1.0$, $UNION = 2.0$, $Nested Subquery = 3.0$).

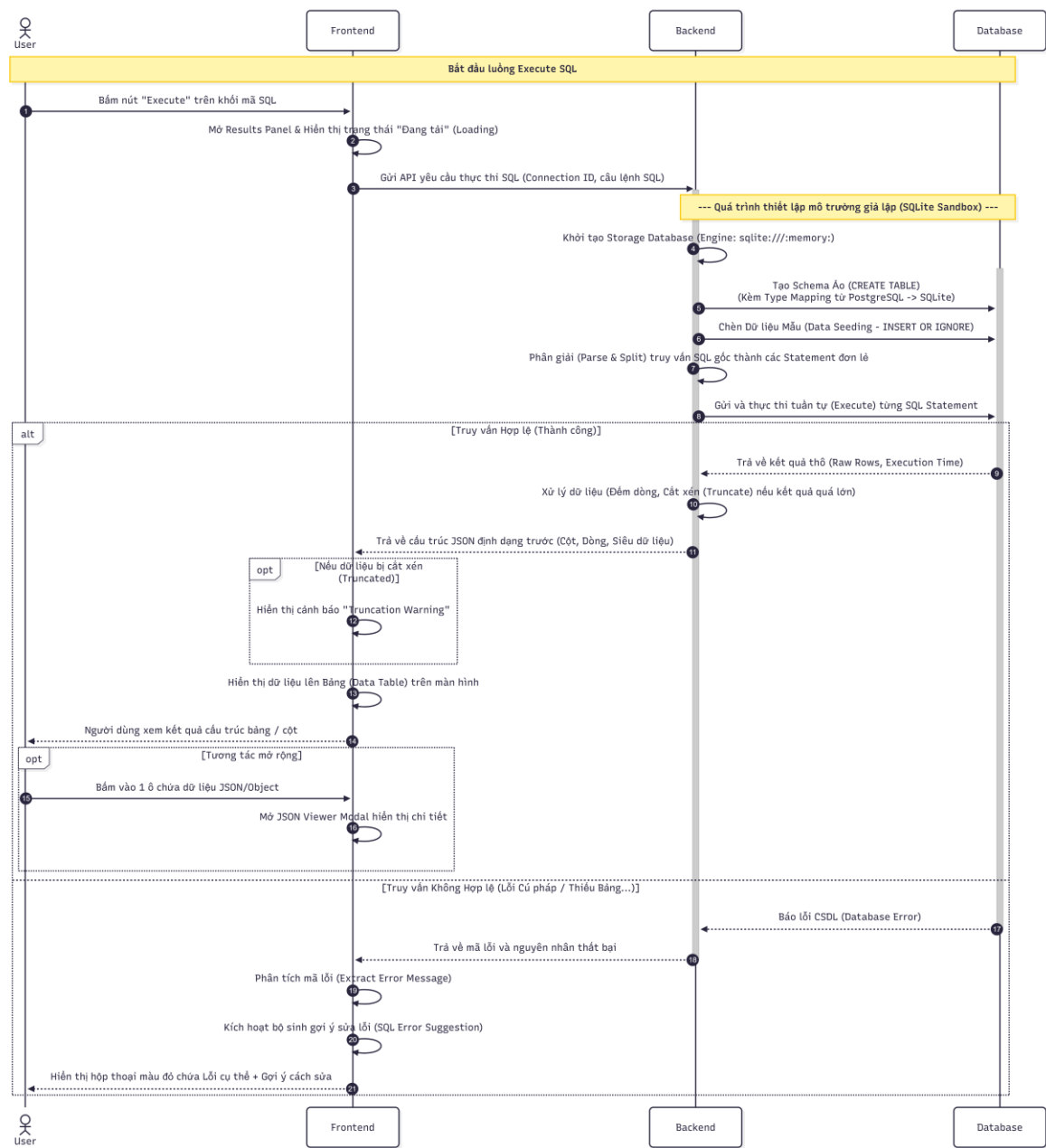
- **Tác dụng:** Dùng để chấm điểm hoặc phân loại mức độ khó của một câu truy vấn dựa trên số lượng hàm và toán tử được sử dụng. Nếu điểm số nằm ở các khoảng cho trước, câu SQL sẽ được gán nhãn Dễ, Vừa, Khó hoặc Cực Khó.
- **Ví dụ:** * Câu SELECT * FROM users không có toán tử phức tạp -> **Mức độ: Dễ**.
 - Câu SELECT sum(price) FROM orders JOIN users GROUP BY user_id có chứa JOIN, SUM và GROUP BY. Mỗi từ khóa được cộng thêm trọng số điểm -> **Mức độ: Khó**.

3. Luồng hoạt động (System Workflow)

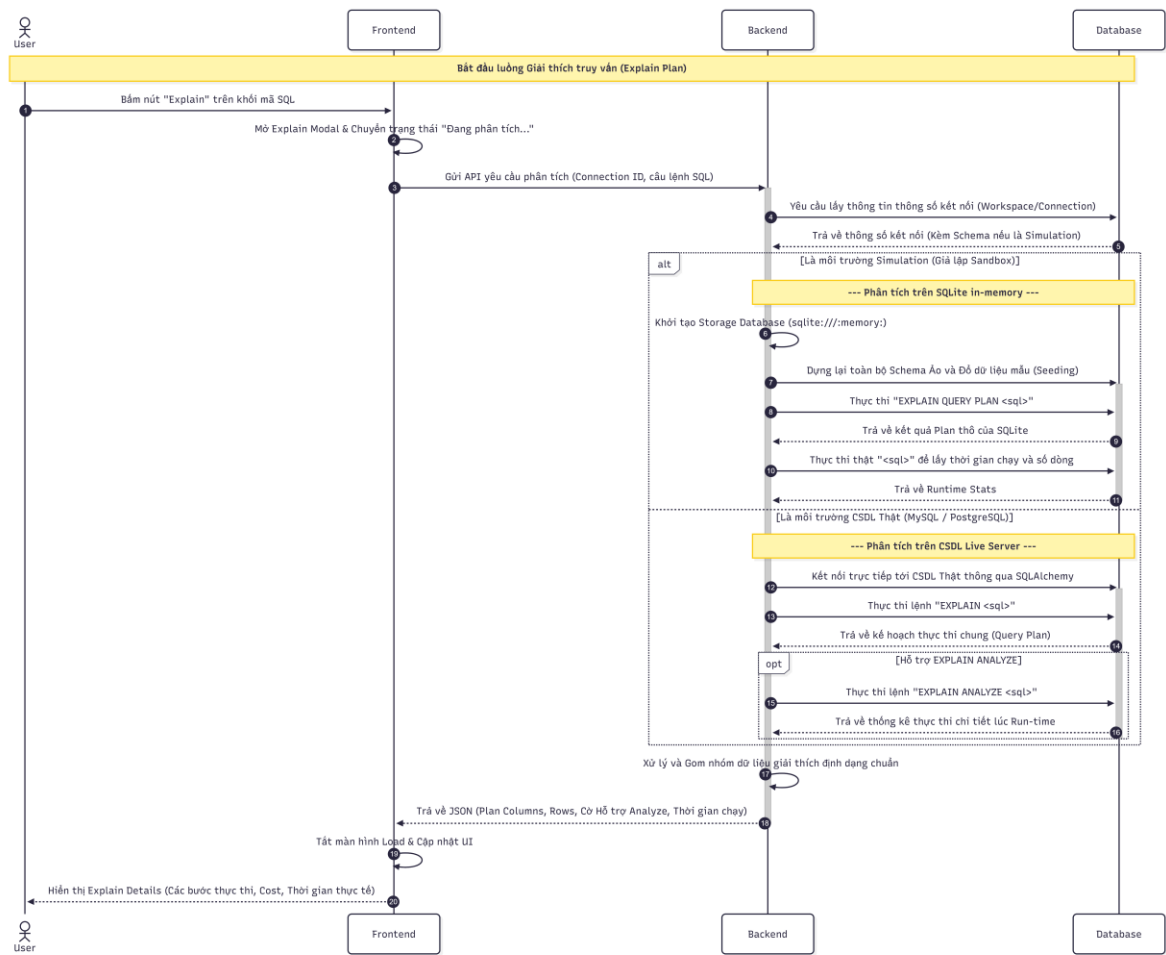
3.1. Text-to-SQL



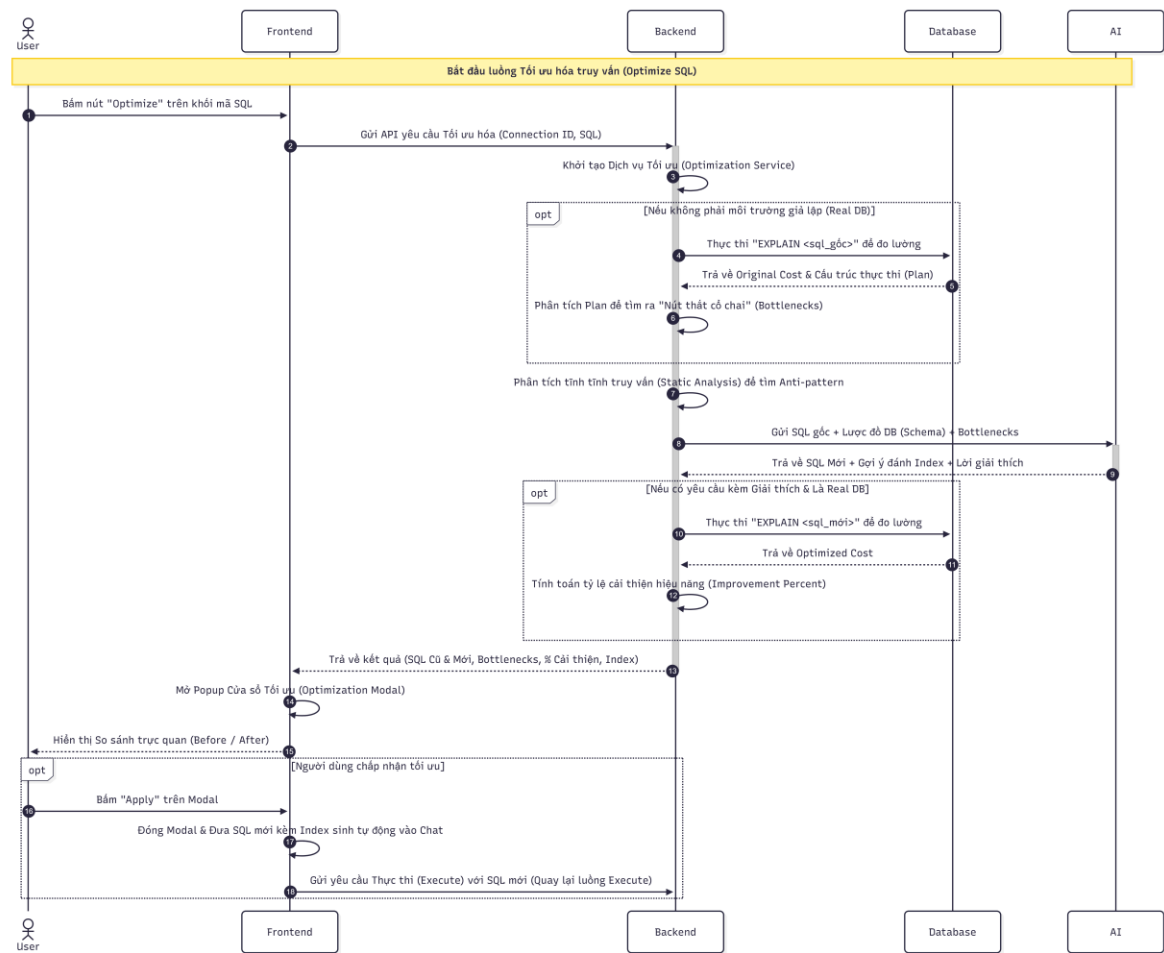
3.2. Excute SQL



3.3. Explain SQL



3.4. Optimize SQL



3.5. Schema design

